PRAMANA

Presentation script

Suvan Sharma (2328069) · Team The Spartan Coders

Guide: Aditya Sharma · Department of CSE · I.K. Gujral Punjab Technical University, Mohali Campus-1

15 slides · about 14 minutes including the live demo

1. How to use this script

- Each slide has three parts: **On screen** (what the audience sees), **Clicks** (what each press reveals, in order) and the **Script**.
- Wherever the script says ▶ **CLICK**, press → or Space once. The click count in each slide matches the deck exactly.
- You do not have to read word for word. Read it aloud twice before the evaluation, then speak from the bold key points.
- The same text is inside the deck as speaker notes: press P during the talk to open the presenter view (notes, timer, current and next slide) on your laptop screen.

Keys during the talk

Key	Action
→ · Space · click	next build / next slide (a presentation clicker works)
←	back
F	full screen
P	presenter view with notes and timer
T	switch dark ↔ light theme (use light if the projector looks washed out)
B	black screen
number + Enter	jump to a slide, e.g. 10 Enter for the multi-agent pipeline

2. Timing plan

Times assume a calm pace of about 140 words a minute. The demo slide includes about 80 seconds for the live run.

#	Section	Slide	Length	Starts at
1	Title	Pramana	0:27	0:00
2	The problem	Legal research today	0:37	0:27
3	The problem	One number, two crimes	0:47	1:04
4	The problem	AI invents law	0:50	1:51
5	What exists	What exists and the gap	1:02	2:41
6	Our solution	Our solution	0:39	3:43
7	Our solution	How we do it	0:41	4:22
8	How it works	System architecture	0:59	5:03
9	How it works	Retrieval	0:50	6:02
10	Multi-agent pipeline	Multi-agent pipeline	1:07	6:52
11	Multi-agent pipeline	Agents 1 and 2: identify and verify	1:06	7:59
12	Multi-agent pipeline	Agents 2b and 4: applicability and defence	1:13	9:05
13	Multi-agent pipeline	Agents 3 and 5: precedents and outcomes	0:57	10:18
14	Technology	Technology stack	0:55	11:15
15	Demo	Live demo	2:04	12:10
		Total	14:14	

If you are running short of time

- **Slide 5 (What exists):** say only the two sentences about tools and the gap.
- **Slide 9 (Retrieval):** skip the four steps' detail; say “BNS 303 comes first: third by text, second by votes.”
- **Slide 15 (Demo):** open the saved sample instead of a live run. It replays instantly.

3. Before you start (10 minutes before)

1. Close Chrome tabs, VS Code and any heavy apps, so the live analysis has memory to work with.
2. Double-click run_web.bat in D:\nayasetu. Wait until the browser opens http://127.0.0.1:8000/ and the bottom-left status shows “Ready”.
3. Open one saved sample (Motorcycle theft) once, so everything is warmed up. Then click “New matter”.
4. Open D:\nayasetu\stuff\presentation\index.html in Chrome or Edge on the projector. Press F for full screen.
5. Press P and drag the presenter window to your laptop screen. Check that the timer runs.
6. Keep the motorcycle facts ready to paste (they are on slide 15 and in section 5 of this script).

4. Slide-by-slide script

Slide 1 · Pramana

Title · about 0:27

On screen: Title, the rotating “Every citation verified” seal, your name, roll number, team, guide and university.

Clicks: none, the slide builds on its own

Script: Good morning. I’m Suvan Sharma from team The Spartan Coders, and my major project is Pramana, an AI-assisted legal research system for Indian criminal law, guided by Aditya Sharma. Pramana is Sanskrit for proof, and that is the design goal: nothing reaches the lawyer unless it can be proved from the statutes and judgments the system actually holds. Let me start with the problem.

Slide 2 · Legal research today

The problem · about 0:37

On screen: Who does legal research, and the four manual steps: bare act → commentary → search judgments → manual reading.

Clicks: 1) the three reasons it is harder now

Script: Legal research in India is mostly manual. Advocates, especially in the district courts, law students and legal-aid clinics find candidate sections in the bare act, read the commentary for the ingredients, search judgments for precedent, and then read for hours. ▶ **CLICK 1** Three things make it harder now: since July 2024 there are two criminal codes in use, AI tools that promise speed invent citations, and precedent is split, because the new code has almost no case law yet. The first of these is the core problem.

Slide 3 · One number, two crimes

The problem · about 0:47

On screen: The IPC and BNS columns with section numbers 302, 303, 420 in the middle.

Clicks: 1) 302: murder vs religious feelings 2) 303: murder by a life-convict vs theft 3) 420: cheating vs no such section 4) “the offence date decides the code”

Script: The Bharatiya Nyaya Sanhita replaced the Indian Penal Code on 1 July 2024, and almost every offence got a new number. Offences committed before that date are still tried under the IPC, so for years every lawyer works with both. ▶ **CLICK 1** 302 is murder in the IPC; in the BNS it is hurting religious feelings. ▶ **CLICK 2** 303 was murder by a life-convict; in the BNS it is theft. ▶ **CLICK 3** And 420, the best-known cheating section, does not exist in the BNS at all. ▶ **CLICK 4** So a citation that is right in one code is a different crime, or nothing, in the other. That is exactly where AI tools go wrong.

Slide 4 · AI invents law

The problem · about 0:50

On screen: A model being asked what BNS 303 covers; it types a wrong answer.

Clicks: 1) the red “Wrong. BNS 303 is Theft” stamp 2) 20 of 68 invented citations 3) 17–33% for commercial tools 4) the problem statement box

Script: The second problem: language models invent citations. Asked what BNS 303 covers, the model confidently described a bomb-threat offence. ▶ **CLICK 1** BNS 303 is theft. ▶ **CLICK 2** When we let it answer from memory on 15 development cases, 20 of its 68 citations named sections that do not exist in the BNS, mostly old IPC numbers. ▶ **CLICK 3** This is not only small models: a Stanford and Yale study published in 2025 found the leading commercial legal AI research tools hallucinating on 17 to 33 percent of queries. ▶ **CLICK 4** So the problem statement: build a research assistant for Indian criminal law that finds the applicable sections and precedents across the transition, and never presents a non-existent citation. What already exists?

Slide 5 · What exists and the gap

What exists · about 1:02

On screen: Five research papers on the left, three groups of existing tools on the right.

Clicks: 1) “The gap” box

Script: On the research side: retrieval-augmented generation grounds a model’s answer in retrieved text, but nothing in it checks that a citation exists. LeSICiN identifies statutes with a graph model, but it is supervised and IPC-only. IL-TUR is the Indian legal benchmark. Magesh and colleagues, at Stanford and Yale, measured hallucination in commercial tools. BGE-M3 is a strong general embedder. On the tool side: Indian Kanoon is a free search engine, and Manupatra and SCC Online are subscription databases; they return real documents but do not map facts to sections. jhana.ai and NyayAssist are AI research assistants that link and check citations, so verification alone is not new. ▶ **CLICK 1** The gap is the combination, which no system we found offers: choosing the code by offence date, graph-verified citations, bridging new BNS charges to old IPC precedent, counter-arguments and outcome statistics. That is what we built.

Slide 6 · Our solution

Our solution · about 0:39

On screen: The motorcycle facts going in on the left.

Clicks: 1) the five-row report coming out 2) the three guarantees

Script: This is our solution, shown with the running example: a motorcycle stolen on 15 August 2024. The lawyer enters the facts and the date. ▶ **CLICK 1** Pramana answers: the BNS applies because of the date; the charge is BNS 303, theft, verified; three precedents, found through its old IPC section; a counter-argument whose quotation is checked word for word; and how 158 appeals under the old theft section were decided. ▶ **CLICK 2** Three guarantees: the right code every time, every citation verified, and precedent across both codes. Here is how we do it.

Slide 7 · How we do it

Our solution · about 0:41

On screen: A four-step track: route, retrieve, verify, ground.

Clicks: 1) 01 Route 2) 02 Retrieve 3) 03 Propose & verify 4) 04 Ground & argue, and “The AI proposes. The graph decides what is real.”

Script: Our approach has four steps. ▶ **CLICK 1** Route: the offence date picks the code, IPC or BNS, with a fixed rule; no AI is involved. ▶ **CLICK 2** Retrieve: find candidate sections, both from statute text and from similar past judgments. ▶ **CLICK 3** Propose and verify: the AI proposes sections, and a citation graph built from the statutes confirms that each one exists in that code. ▶ **CLICK 4** Ground and argue: test each section against the facts, find precedents, and draft counter-arguments. The principle behind it: the AI proposes, the graph decides what is real. Here is the full architecture.

Slide 8 · System architecture

How it works · about 0:59

On screen: The full animated architecture; a pulse travels the pipeline.

Clicks: 1) the legend and the three stores built once

Script: This is the full architecture. Solid boxes are ordinary code, dashed boxes are the language model, and green boxes are lookups in the citation graph. A case flows left to right along the top: the router picks the code, retrieval finds candidates, a guardrail stops early if nothing is close, Node 1 proposes sections and Node 2 verifies them, sending rejections back. Then down the right and back along the bottom: Node 2b tests whether each real section applies, Node 3 finds precedents and Node 5 counts outcomes in parallel, Node 4 drafts counter-arguments, and a final check re-verifies everything before the report. ▶ **CLICK 1** Three stores are built once from public data: the IPC, the BNS from the official gazette, 2,699 judgments and the IPC-to-BNS mapping. Here is how a case finds its candidate sections in them.

Slide 9 · Retrieval

How it works · about 0:50

On screen: Why RAG on the left; the motorcycle case step by step on the right.

Clicks: 1) 1 Date rule → BNS 2) 2 Embed the facts 3) 3 Statute search + precedent votes 4) 4 the 14 candidates, BNS 303 first, and the guardrail line

Script: Why retrieval-augmented generation and not fine-tuning? The law keeps changing, and with retrieval new judgments are simply added. Every candidate traces back to a real indexed section. And the model reads the real section text before it proposes anything. Here is the motorcycle case. ▶ **CLICK 1** The date rule picks the BNS. ▶ **CLICK 2** BGE-M3 turns the facts into a 1024-dimensional vector. ▶ **CLICK 3** Two rankings are built: similarity to statute text, and votes from similar judgments for the sections they applied, combined by reciprocal rank fusion. ▶ **CLICK 4** BNS 303, theft, comes first: third by text and second by votes. A guardrail checks that the closest judgment is near enough before the agents are asked. Those agents are next.

Slide 10 · Multi-agent pipeline Multi-agent pipeline · about 1:07

On screen: The shared-state box in the centre.

Clicks: 1) agents 1 and 2 with the retry loop 2) agent 2b 3) agents 3 and 5 4) agent 4

Script: The pipeline is a set of agents built with LangGraph, all reading and writing one shared, typed state: the facts, the date, the code, the candidates, what was verified or rejected, the precedents and the defences. ▶ **CLICK 1** The statute identifier, a language model, proposes sections from the facts and the candidates. The verifier checks each one in the citation graph; if any is rejected, the case goes back to the identifier with the reason, at most twice. ▶ **CLICK 2** The applicability judge, a language model guided by fixed legal rules, tests whether each real section fits the facts. ▶ **CLICK 3** Then two agents run in parallel: the precedent matcher walks the citation graph, including the bridge from BNS to IPC, and the outcome analyst counts how appeals were decided with plain SQL. ▶ **CLICK 4** Finally the defence drafter writes counter-arguments, and a final check verifies everything it cites. Let me take the agents one at a time, starting with verification.

Slide 11 · Agents 1 and 2: identify and verify Multi-agent pipeline · about 1:06

On screen: The real citation graph around theft on the left.

Clicks: 1) Round 0: 421 rejected 2) Round 1: 378 rejected, hint 303 3) Round 2: 303 verified 4) 20/20 and 0

Script: Agents one and two. The citation graph holds every section of both codes, every judgment, and an edge whenever a judgment cites a section; on the left is a real piece of it around theft. The rule is one line: a section is real only if it is a node of the code the router chose. In the normal motorcycle run, BNS 303 verifies at once. To show the loop, this is a stress-test run with retrieval withheld. ▶ **CLICK 1** Round 0: the identifier says 421. Rejected: the BNS stops at 358. ▶ **CLICK 2** Round 1: it says 378, the old IPC theft section. Rejected, but the graph supplies a hint: its BNS counterpart is 303. ▶ **CLICK 3** Round 2: 303, verified. ▶ **CLICK 4** The graph tells real section numbers from invented ones 20 out of 20 times, and no rejected citation has ever reached a report. But a real section can still be the wrong charge.

Slide 12 · Agents 2b and 4: applicability and defence Multi-agent pipeline · about 1:13

On screen: The knife-stabbing facts on the left.

Clicks: 1) the facts highlight as evidence 2) 109 and 118(2) supported 3) 105 doubted (no death), 117 included, legal rules 4) agent 4: three checked counter-arguments

Script: Existing is not the same as applying, so agent 2b, the applicability judge, tests every verified section against the facts. Here is a different saved case: a stabbing after a parking quarrel. ▶ **CLICK 1** The judge must quote the facts for each ingredient: a kitchen knife, a stab to the abdomen, a perforated intestine, an injury dangerous to life. ▶ **CLICK 2** So attempt to murder and hurt by a dangerous weapon are supported. ▶ **CLICK 3** Culpable homicide is a real section, but it is doubted, because it requires a death and the facts report grievous hurt. Fixed legal rules back the judge: a death for homicide, five people for rioting, a lesser

offence included in a greater one. Grievous hurt is shown as included in the weapon charge. Nothing is hidden; doubts are shown with their reason. ▶ **CLICK 4** Agent four then drafts three counter-arguments, and a final check confirms each one targets a charge, quotes the facts word for word, and cites only precedents it was given. Where do those precedents come from?

Slide 13 · Agents 3 and 5: precedents and outcomes

Multi-agent pipeline · about 0:57

On screen: BNS 303 alone on the graph.

Clicks: 1) BNS 303: 0 judgments 2) IPC 378: 11 and IPC 379: 158 through the mapping 3) the top three precedents 4) agent 5's outcome counts

Script: Agent three finds precedents only through graph edges, so every precedent really did apply the charged section. ▶ **CLICK 1** For BNS 303, not one judgment in our corpus applies it yet; BNS theft cases have not reached the higher courts in bulk. ▶ **CLICK 2** So the agent crosses the reviewed mapping to the old IPC sections: 378 with 11 judgments and 379 with 158. The law of theft did not change, so that precedent still counts. ▶ **CLICK 3** The top three are ranked by similarity to the facts. ▶ **CLICK 4** In parallel, agent five counts how appeals under each section were decided, with plain SQL and no language model: under IPC 379, 158 judgments, 76 allowed and 76 dismissed. Anything under five judgments is not shown. That is every agent; here is the technology behind them.

Slide 14 · Technology stack

Technology · about 0:55

On screen: The four-layer stack with the reason for each choice, and the wall of tools.

Clicks: none, the slide builds on its own

Script: The technology stack, layer by layer, with the reason for each choice. The knowledge layer: ChromaDB holds the vectors, persistent and filterable by metadata; NetworkX holds the citation graph, which gives an exact existence check; SQLite holds the outcome counts. The intelligence layer: BGE-M3 for embeddings, chosen by a measured comparison against a legal-domain model, and qwen 3.5, an open-weight 4-billion-parameter model served through Ollama, whose answers are constrained to a JSON schema. Orchestration is LangGraph rather than one big prompt, because we need the retry edge, parallel agents and a state we can check after every step. The interface is FastAPI with a web app that shows each stage live. On the right, every tool in the project, with 422 automated tests. Let me show it running.

Slide 15 · Live demo · Thank you

Demo · about 2:04

On screen: The case card and the “Switching to the browser” frame.

Clicks: 1) “Thank you” (press after the demo)

Script: Switch to the browser: run_web.bat, then 127.0.0.1:8000. Paste the motorcycle facts; the date is read from the text. While it runs, point out the router choosing BNS, 303 ranked first, the graph check, and precedents arriving through IPC 379. The report opens with its result summary. If anything goes wrong live,

open the saved Motorcycle theft sample, which replays instantly. ▶ **CLICK 1** Back on the slide: Pramana makes one promise and keeps it. Every section it shows exists in the code the offence date selects, and when it doubts that a real section applies, it says so. Thank you; I'm happy to take questions.

5. Live demo runbook (slide 15)

Facts to paste:

> *On 15 August 2024 the complainant parked his motorcycle outside his house at night. In the morning it was gone. CCTV shows the accused pushing it away down the lane and riding off. The accused had no permission to take it and did not return it.*

1. Say “Let’s run it” and switch to the browser (Alt+Tab).
2. Paste the facts into the box and press Analyse. The date is read from the text automatically.
3. While it runs (about a minute), point at the screen as each stage lights up: “the router picked the BNS” → “BNS 303 is ranked first” → “the citation graph verified it” → “precedents arrive through IPC 379”.
4. When the report opens, read the summary line: “1 charge supported: BNS 303, Theft.” Click the BNS 303 pill to show the bare act, then close it.
5. Scroll to Precedents and Counter-arguments briefly, then switch back to the slides and press → for “Thank you”.

If something goes wrong

Problem	What to do
The live run is slow or stuck	Say “while that runs, here is a saved run of the same case”, click “Motorcycle theft” in the left sidebar. It replays instantly.
The page does not open	Double-click run_web.bat again; it reopens the page if the server is already running.
The status says the local model is not running	Use the saved samples; they work without the model.
The browser itself fails	Stay on the slides: slide 15’s case card and slides 11–13 already show real outputs.
The projector washes out the dark slides	Press T for the light theme.

6. Likely questions and answers

Answer in two or three sentences, then stop. If a question needs a slide, type its number and press Enter.

Q1. Why not fine-tune a model on Indian law?
Retrieval keeps the law current: new judgments are added, not trained in. Every answer traces to a real indexed section. And fine-tuning does not stop a model inventing citations; our verification layer does. Fine-tuning alone also gains little: the fine-tuned Indian legal model AALAP scored 25.56% on the bar exam against 23.48% for its base model, and both failed.

Q2. How do you know a citation is real?
Every section the model proposes is looked up in a citation graph built from the actual IPC and BNS text, in the code the offence date selects. It passed a test of 10 real and 10 invented section numbers, including traps like BNS 420, 20 out of 20. Across all our evaluations, no invented or wrong-code section reached a report.

Q3. What happens when the model gets a section wrong?
The verifier rejects it, and the case goes back to the model with the reason and a hint from the graph, at most twice. In the stress test, 421 was rejected, then 378, then 303 was verified. Every affected case recovered within two retries.

Q4. How is this different from jhana.ai or Indian Kanoon?
Indian Kanoon and the databases are search tools; they do not map facts to sections. AI assistants like jhana.ai and NyayAssist do check citations, so verification alone is not new. What we combine is the code chosen by offence date, graph-verified citations, the IPC-to-BNS precedent bridge, counter-arguments and outcome statistics.

Q5. Why is the score on IL-TUR lower than other systems?
Those systems are classifiers trained on 42,000 labelled documents for that one benchmark. Pramana has no training on the task and also does routing, precedents and arguments. It scores 11.8 macro-F1 against 28.1; it is strong on serious offences, for example murder at 0.78, and the gap is mainly minor companion offences.

Q6. Where do precedents for new BNS sections come from?
Almost no BNS case has reached the higher courts yet. So Pramana crosses a reviewed 575-row IPC-to-BNS mapping: for BNS 303, theft, there are 0 direct judgments but 11 under IPC 378 and 158 under IPC 379.

Q7. Why LangGraph and multiple agents, not one prompt?
The pipeline needs a conditional retry edge, two agents running in parallel, and a shared state where every step’s output can be checked on its own. A single prompt gives none of that.

Q8. Why BGE-M3 and not a legal embedding model?
We measured it. On 15 test cases BGE-M3 found a relevant section in the top 3 for 0.533 of cases against 0.20 for the legal-domain model, which kept returning the same generic sections for every query.

Q9. What data did you use?
All 575 IPC sections, all 358 BNS sections parsed from the official gazette, 2,699 Supreme Court and High Court judgments split into 87,036 passages, and the reviewed IPC-to-BNS mapping. The citation graph has 933 sections and 16,493 citation edges.

Q10. How did you evaluate it fairly?

Decisions were made only on development sets. Held-out sets were written before each change and scored once. Real High Court orders were never read before scoring, and the IL-TUR test split was run once at the very end.

Q11. What does the outcome statistic mean?

It counts how appeals citing a section were decided, allowed or dismissed, using SQL over the judgments; no model generates the numbers. It is an appeal outcome, not a conviction rate, and anything under five judgments is not shown.

Q12. What are the limitations?

Choosing exactly the right sections is the hard part: sometimes extra sections are proposed, which the applicability check flags as doubted. Minor offences are often missed, the mapping has not yet been reviewed by practising lawyers, and it covers substantive criminal law in English.

Q13. What would you do next?

Widen retrieval for minor offences, have lawyers review the mapping, ingest new BNS judgments as they are published, and extend to the new procedure and evidence codes, Hindi input and scanned documents.

7. Key numbers to remember

Fact	Number
Invented or wrong-code sections that reached a report	0
Citation test: real vs invented section numbers	20 / 20
Routing: correct code	30 / 30
Citations the model invented from memory	20 of 68
Cases recovered within two retries	6 / 6
IPC sections · BNS sections	575 · 358
Judgments · indexed passages	2,699 · 87,036
Citation edges · IPC→BNS bridges	16,493 · 539
BNS 303 direct precedents · via IPC 378 · via IPC 379	0 · 11 · 158
IL-TUR macro-F1: Pramana · best supervised	11.8 · 28.1
Embedder recall@3: BGE-M3 · legal model	0.533 · 0.20
Automated tests	422
Commercial legal AI tools hallucinating (Magesh et al.)	17–33%